

Blatt 9

Vorkurs Bauingenieurwesen - Informatik - 06.09.24

Aufgabe 0

Erstelle eine neue Textdatei, die auf `.py` endet, etwa mit dem Namen `blatt09.py`. In diese Datei sollen die Lösungen der folgenden Aufgaben geschrieben werden.

Aufgabe 1

Schreibe an den Anfang der Datei aus Aufgabe 0 `import sortieren` und speichere die Datei `sortieren.py` von GitHub direkt im selben Ordner wie die aktuelle Datei.

Definiere eine globale Variable `liste` (also einfach lose in der Datei, nicht in einer Funktion), die als Wert eine Liste von mindestens zehn ganzen Zahlen hat, nicht alle gleich.

Aufgabe 2

Schreibe eine Funktion `groesser`, die zwei ganze Zahlen `x` und `y` übergeben bekommt und zurück gibt, ob `x` größer als `y` ist.

```
def groesser(x: int, y: int) -> bool:
```

Nutze die Funktion `sortieren.mergesort_mit`, um die Variable `liste` aus Aufgabe 1 absteigend zu sortieren. Übergebe dafür `groesser` als erstes Argument von `sortieren.mergesort_mit`.

Aufgabe 3

Direkt in python enthalten ist die `abs` Funktion. Sie gibt den Betrag des Argumentes zurück. Schreibe eine Funktion `kleinerer_betrag`, die zwei ganze Zahlen `x` und `y` übergeben bekommt und zurück gibt, ob `x` näher an 0 als `y` ist.

```
def kleinerer_betrag(x: int, y: int) -> bool:
```

Nutze die Funktion `sortieren.mergesort_mit`, um die Variable `liste` aus Aufgabe 1 nach Betrag zu sortieren. Übergebe dafür `kleinerer_betrag` als erstes Argument von `sortieren.mergesort_mit`.

Aufgabe 4

Lass `sortieren.mergesort_mit` die Liste `liste` aus Aufgabe 1 nach Abstand zur Zahl 5 sortieren. Also 5 ist am kleinsten, 4 und 6 sind am zweitkleinsten, 3 und 7 kommen danach, etc. Schreibe dazu wieder

eine Funktion, die zwei Zahlen `x` und `y` übergeben bekommt und zurückgibt, ob `x` in diesem Sinne *kleiner* als `y` ist.

Aufgabe 5

Schreibe eine Funktion `kleinster_teiler`, die eine ganze Zahl `x` übergeben bekommt und den kleinsten Teiler von `x` größer als 1 zurück gibt. Als Ausnahme soll 1 zurückgegeben werden, wenn `x` selber 1 ist. Zum Beispiel soll `kleinster_teiler(12)` den Wert 2 zurückgeben, `kleinster_teiler(13)` den Wert 13 und `kleinster_teiler(1)` den Wert 1. Entscheide selber, wie negative Zahlen behandelt werden sollen.

```
def kleinster_teiler(x: int) -> int:
```

Tipp: `x % y == 0` testet, ob `y` ein Teiler von `x` ist. Im Detail kann die Funktion so umgesetzt werden:

1. für alle `y` von 2 bis `x`:
 - (a) wenn `y` ein Teiler von `x` ist, gebe `y` zurück.
2. gebe `x` zurück

Aufgabe 6

Lass `sortieren.mergesort_mit` die Variable `liste` nach kleinstem Teiler sortieren. Also 1 ist am kleinsten, durch 2 teilbare Zahlen am zweitkleinsten, ungerade aber durch 3 teilbare Zahlen am drittkleinsten, etc.

Schreibe dazu wieder eine Funktion, die zwei Zahlen `x` und `y` übergeben bekommt und zurückgibt, ob `x` in diesem Sinne *kleiner* als `y` ist.

Bonus: wenn `x` und `y` den selben kleinsten Primfaktor haben, vergleiche den zweitkleinsten, dann den dritt-kleinsten, etc.

Aufgabe 7

Lass `sortieren.mergesort_mit` die Variable `liste` aus Aufgabe 1 so sortieren, dass Primzahlen kleiner als zusammengesetzte Zahlen sind.

Aufgabe 8

Denke dir eine kurze Liste von *cool*en Zahlen aus. Lass `sortieren.mergesort_mit` die Variable `liste` aus Aufgabe 1 so sortieren, dass die coolen Zahlen an den Anfang sortiert werden.